實作題

第 2 題 矩陣轉換

問題描述

矩陣是將一群元素整齊的排列成一個矩形，在矩陣中的橫排稱為列 (row)，直排稱為行 (column)，其中以 X_{ij} 來表示矩陣 X 中的第 i 列第 j 行的元素。如圖一中，$X_{32} = 6$。

我們可以對矩陣定義兩種操作如下：

翻轉：即第一列與最後一列交換、第二列與倒數第二列交換、...依此類推。

旋轉：將矩陣以順時針方向轉 90 度。

例如：矩陣 X 翻轉後可得到 Y，將矩陣 Y 再旋轉後可得到 Z。

X

1	4
2	5
3	6

Y

3	6
2	5
1	4

Z

1	2	3
4	5	6

圖一

一個矩陣 A 可以經過一連串的旋轉與翻轉操作後，轉換成新矩陣 B。如圖二中，A 經過翻轉與兩次旋轉後，可以得到 B。給定矩陣 B 和一連串的操作，請算出原始的矩陣 A。

A

1	1
1	3
2	1

翻轉 →

2	1
1	3
1	1

旋轉 →

1	1	2
1	3	1

旋轉 → B

1	1
3	1
1	2

圖二

輸入格式

第一行有三個介於 1 與 10 之間的正整數 R, C, M。接下來有 R 行(line)是矩陣 B 的內容，每一行(line)都包含 C 個正整數，其中的第 i 行第 j 個數字代表矩陣 B_{ij} 的值。在矩陣內容後的一行有 M 個整數，表示對矩陣 A 進行的操作。第 k 個整數 m_k 代表第 k 個操作，如果 $m_k = 0$ 則代表旋轉，$m_k = 1$ 代表翻轉。同一行的數字之間都是以一個空白間格，且矩陣內容為 0~9 的整數。

輸出格式

輸出包含兩個部分。第一個部分有一行，包含兩個正整數 R' 和 C'，以一個空白隔開，分別代表矩陣 A 的列數和行數。接下來有 R' 行，每一行都包含 C' 個正整數，且每一行的整數之間以一個空白隔開，其中第 i 行的第 j 個數字代表矩陣 A_{ij} 的值。每一行的最後一個數字後並無空白。

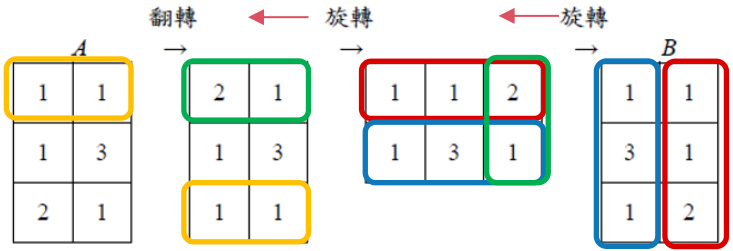
範例一：輸入

```
3 2 3
1 1
3 1
1 2
1 0 0
```

範例一：正確輸出

```
3 2
1 1
1 3
2 1
```

（說明）



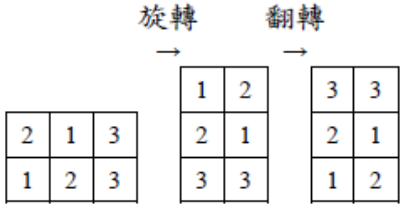
範例二：輸入

```
3 2 2
3 3
2 1
1 2
0 1
```

範例二：正確輸出

```
2 3
2 1 3
1 2 3
```

（說明）



評分說明

輸入包含若干筆測試資料，每一筆測試資料的執行時間限制(time limit)均為 2 秒，依正確通過測資筆數給分。其中：

第一子題組共 30 分，其每個操作都是翻轉。

第二子題組共 70 分，操作有翻轉也有旋轉。

解題思路

讀取資料

R C M :列 欄 操作次數
讀取R次，切割為C筆資料，加入二維List
讀取操作，切割為 M 筆操作

操作資料

反向順序

0 -> 旋轉(rotate) / 1 -> 翻轉(flip)

輸出

輸出 R C

輸出二維List:輸出R次，每列有C筆資料

10503P2.py - D:\APCSClass3\Solutions\10503\10503P...

File Edit Format Run Options Window Help

```python
def rotate(matrixB):
    matrixA = []
    r = len(matrixB)
    c = len(matrixB[0])
    for i in range(c-1, -1, -1):
        row = []
        for j in range(r):
            row.append(matrixB[j][i])
        matrixA.append(row)
    return matrixA

def flip(matrixB):
    matrixA = []
    r = len(matrixB)
    for i in range(r-1, -1, -1):
        matrixA.append(matrixB[i])
    return matrixA
```

注意方向

```python
#讀取資料 R C M：列 欄 操作次數
inStrs = input().split(' ')
R = int(inStrs[0])
C = int(inStrs[1])
M = int(inStrs[2])

#讀取R次，切割為C筆資料，加入二維List
matrix = []
for i in range(R):
    temp = input().split(' ')
    rList = []
    for j in range(C):
        rList.append(int(temp[j]))
    matrix.append(rList)

#讀取操作，切割為 M 筆操作
ops = input().split(' ')
```

```python
#操作資料 : 0->旋轉(rotate) / 1->翻轉(flip)
#反向順序
for k in range(M-1, -1, -1):
    op = int(ops[k])
    if(op==0):
        matrix = rotate(matrix)
        #print('rotate', matrix)
    elif(op==1):
        matrix = flip(matrix)
        #print('flip', matrix)

#輸出 R C
Rr = len(matrix)
Rc = len(matrix[0])
print(Rr, Rc)

#輸出二維List：輸出R次，每列有C筆資料
for i in range(Rr):
    for j in range(Rc):
        print(matrix[i][j], end=' ')
    print()
```

Ln: 43 Col: 9

IDLE Shell 3.9.7

File Edit Shell Debug Options Window Help

```
>>>
= RESTART: D:\APCSClass3\Solutions\10503\10503P2.py
3 2 3
1 1
3 1
1 2
1 0 0
3 2
1 1
1 3
2 1
>>>
= RESTART: D:\APCSClass3\Solutions\10503\10503P2.py
3 2 2
3 3
2 1
1 2
0 1

2 3
2 1 3
1 2 3
>>>
```

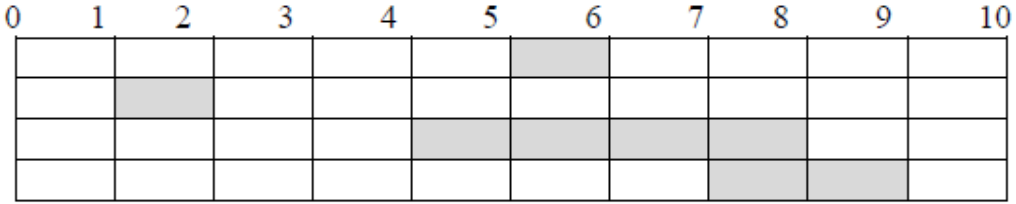
Ln: 23 Col: 4

實作題

第 3 題 線段覆蓋長度

問題描述

給定一維座標上一些線段，求這些線段所覆蓋的長度，注意，重疊的部分只能算一次。例如給定三個線段：(5, 6)、(1, 2)、(4, 8)、和(7, 9)，如下圖，線段覆蓋長度為 6。



輸入格式：

第一列是一個正整數 N，表示此測試案例有 N 個線段。

接著的 N 列每一列是一個線段的開始端點座標和結束端點座標整數值，開始端點座標值小於等於結束端點座標值，兩者之間以一個空格區隔。

輸出格式：

輸出其總覆蓋的長度 。

範例一：輸入

輸入	說明
5	此測試案例有 5 個線段
160 180	開始端點座標值與結束端點座標
150 200	開始端點座標值與結束端點座標
280 300	開始端點座標值與結束端點座標
300 330	開始端點座標值與結束端點座標
190 210	開始端點座標值與結束端點座標

範例一：輸出

輸出	說明
110	測試案例的結果

160~180 可以被 150~200取代

190~210可取代 150~200 改為150~210

範例二：輸入

輸入	說明
1	此測試案例有 1 個線段
120 120	開始端點座標值與結束端點座標值

範例二：輸出

輸出	說明
0	測試案例的結果

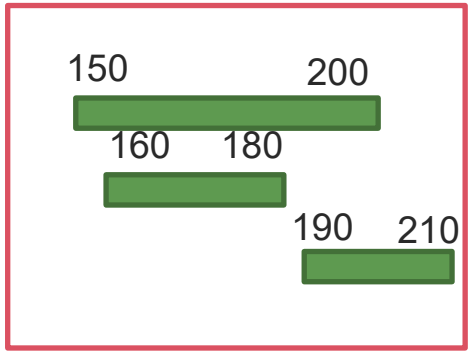
評分說明

輸入包含若干筆測試資料，每一筆測試資料的執行時間限制(time limit)均為 2 秒，依正確通過測資筆數給分。每一個端點座標是一個介於 0~M 之間的整數，每筆測試案例線段個數上限為 N。其中：

第一子題組共 30 分，M<1000，N<100，線段沒有重疊。

第二子題組共 40 分，M<1000，N<100，線段可能重疊。

第三子題組共 30 分，M<10000000，N<10000，線段可能重疊。

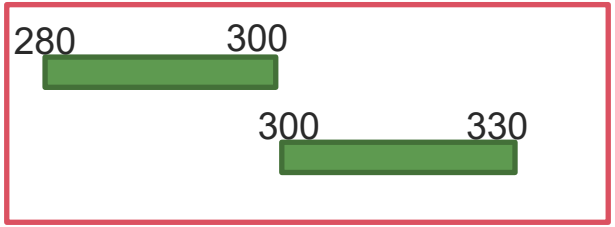


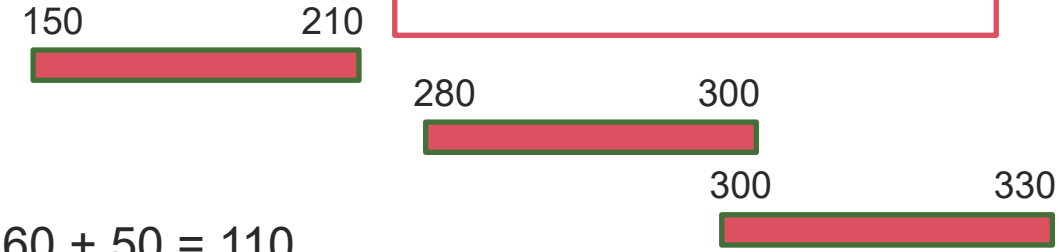
1. 如果A的尾巴 大於 B的頭（A尾 > B頭），
就要判斷有沒有真的需要合併：

1-1. 如果B的尾巴 <= A的尾巴
B線段完全被A包含，不用動，忽略B。
口訣：「B被A蓋住，B不用理。」

1-2. 如果B的尾巴 > A的尾巴
A跟B有重疊，需要把尾巴延長到B的尾巴。
口訣：「B尾巴比A長，A尾巴跟著變長。」

2. 如果A的尾巴 <= B的頭
A和B沒重疊，B是獨立新段。
口訣：「沒碰到，就分開，開始新的一段。」





60 + 50 = 110

150	200
160	180
190	210
280	300
300	330

10503P3.py - D:\APCS\APCS考古題\10503P3.py (3.8.1)

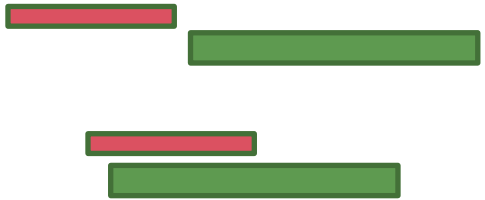
File Edit Format Run Options Window Help

```python
#讀取線段數量 n
n = int(input())

#讀取n次字串，切割組成線段起點及終點列表，加入 lines 二維列表
lines=[]
for i in range(n):
    pairs = input().split(' ')
    lines.append([int(pairs[0]), int(pairs[1])])

#將 lines 以線段起點排序
lines.sort(key=lambda x:x[0])
#print(lines)
```

```python
#將線段合併處理為不重疊線段，加入resultLines二維列表
#以第一組線段(lines[0])作為當前線段curline
resultLines=[]
curline = lines[0]
#將lines[i] 由索引1-n 逐一取出
for i in range(1, n):
    #若當前線段與lines[i]沒有重疊，當前線段加入resultLines
    if(curline[1]<lines[i][0]):
        resultLines.append(curline)
        curline = lines[i]
    #若當前線段與lines[i] 重疊且curline終點<lines[i]終點，合併線段
    elif(curline[1]<lines[i][1]):
        curline = [curline[0], lines[i][1]]
#將最後的當前線段加入resultLines
resultLines.append(curline)
#print(resultLines)
```

```python
#輸出不重疊線段總長度
length = 0
for rLine in resultLines:
    length = length + (rLine[1]-rLine[0])

print(length)
```

因為排序所以現在起點一定小於下一個起點



實作題

第 2 題　小群體

問題描述

Q 同學正在學習程式，P 老師出了以下的題目讓他練習。

一群人在一起時經常會形成一個一個的小群體。假設有 N 個人，編號由 0 到 N-1，每個人都寫下他最好朋友的編號（最好朋友有可能是他自己的編號，如果他自己沒有其他好友），在本題中，**每個人的好友編號絕對不會重複，也就是說 0 到 N-1 每個數字都恰好出現一次**。

這種好友的關係會形成一些小群體。例如 N=10，好友編號如下，

	0	1	2	3	4	5	6	7	8	9
好友編號	4	7	2	9	6	0	8	1	5	3

0 的好友是 4，4 的好友是 6，6 的好友是 8，8 的好友是 5，5 的好友是 0，所以 0、4、6、8、和 5 就形成了一個小群體。另外，1 的好友是 7 而且 7 的好友是 1，所以 1 和 7 形成另一個小群體，同理，3 和 9 是一個小群體，而 2 的好友是自己，因此他自己是一個小群體。總而言之，在這個例子裡有 4 個小群體：{0,4,6,8,5}、{1,7}、{3,9}、{2}。本題的問題是：輸入每個人的好友編號，計算出總共有幾個小群體。

Q 同學想了想卻不知如何下手，和藹可親的 P 老師於是給了他以下的提示：如果你從任何一人 x 開始，追蹤他的好友，好友的好友，....，這樣一直下去，一定會形成一個圈回到 x，這就是一個小群體。如果我們追蹤的過程中把追蹤過的加以標記，很容易知道哪些人已經追蹤過，因此，當一個小群體找到之後，我們再從任何一個還未追蹤過的開始繼續找下一個小群體，直到所有的人都追蹤完畢。

Q 同學聽完之後很順利的完成了作業。

在本題中，你的任務與 Q 同學一樣：給定一群人的好友，請計算出小群體個數。

輸入格式

第一行是一個正整數 N，說明團體中人數。

第二行依序是 0 的好友編號、1 的好友編號、……、N-1 的好友編號。共有 N 個數字，包含 0 到 N-1 的每個數字恰好出現一次，數字間會有一個空白隔開。

輸出格式

請輸出小群體的個數。不要有任何多餘的字或空白，並以換行字元結尾。

範例一：輸入

10
4 7 2 9 6 0 8 1 5 3

範例一：正確輸出

4

（說明）

4 個小群體是{0,4,6,8,5}, {1,7}, {3,9}和 {2}。

範例二：輸入

3
0 2 1

範例二：正確輸出

2

（說明）

2 個小群體分別是{0},{1,2}。

評分說明

輸入包含若干筆測試資料，每一筆測試資料的執行時間限制(time limit)均為 1 秒，依正確通過測資筆數給分。其中：

第 1 子題組 20 分，$1 \le N \le 100$，每一個小群體不超過 2 人。

第 2 子題組 30 分，$1 \le N \le 1{,}000$，無其他限制。

第 3 子題組 50 分，$1{,}001 \le N \le 50{,}000$，無其他限制。

解題思路

- 讀取
  - 總群體人數N及最好朋友字串，切割轉型後存入集合BF
- 小群體G (二維List / List of Set) 搜尋處理
  - 宣告列表G，包含多個集合LG
  - 索引i由0-N，驗證i是否存在於已知LG中，存在則跳離迴圈
  - 不存在則建立小群體 newLG，加入第一個人(索引i)
    - 取得好友 f = BF[i]， f 不等於小群體中第一個人newLG[0]，索引 f 加入newLG
    - 取得下一個好友f = BF[f]，重複執行直到 f 為小群體中第一個人newLG[0] 為止
  - newLG 加入列表G
- 輸出G 中 LG 的組數

```python
N = int(input())                          #讀取總群體人數N
BF = list(map(int, input().split()))      #最好朋友字串，切割轉型後存入集合BF
#print(BF)

'''小群體G (二維List) 搜尋處理'''

G = []                                    #宣告列表G，包含多個集合LG
for i in range(N):                        #索引i由0-N，驗證i是否存在於已知LG中
    for LG in G:
        if(i in LG):                      #i存在於其中一個小群體:跳離迴圈
            break
    else:                                 #i不存在於目前小群體:建立newLG，索引i加入newLG
        newLG = []
        newLG.append(i)
        f = BF[i]                         # 取得好友 f = BF[i]

        while f != newLG[0]:              # f 不等於newLG[0]，索引 f 加入newLG
            newLG.append(f)
            f = BF[f]                     # 取得下一個好友f = BF[f]，重複執行
            #print(newLG)
        G.append(newLG)                   # newLG 加入列表G

print(len(G))                             # 輸出G 中 LG 的組數
#print(G)
```

實作題

第 2 題　最大和

問題描述

給定 N 群數字，每群都恰有 M 個正整數。若從每群數字中各選擇一個數字 (假設第 i 群所選出數字為 t_i)，將所選出的 N 個數字加總即可得總和 $S = t_1+t_2+\ldots+t_N$。請寫程式計算 S 的最大值 (最大總和)，並判斷各群所選出的數字是否可以整除 S。

評分說明

輸入包含若干筆測試資料，每一筆測試資料的執行時間限制(time limit)均為 1 秒，依正確通過測資筆數給分。其中：

第 1 子題組 20 分：$1 \leqq N \leqq 20$，$M = 1$。

第 2 子題組 30 分：$1 \leqq N \leqq 20$，$M = 2$。

第 3 子題組 50 分：$1 \leqq N \leqq 20$，$1 \leqq M \leqq 20$。

輸入格式

第一行有二個正整數 N 和 M， 1≦ N ≦ 20，1≦ M ≦ 20。

接下來的 N 行，每一行各有 M 個正整數 x_i ，代表一群整數，數字與數字間有一個空格，且 1≦ i ≦M，以及 1≦ x_i ≦256。

輸出格式

第一行輸出最大總和 S。

第二行按照被選擇數字所屬群的順序，輸出可以整除 S 的被選擇數字，數字與數字間以一個空格隔開，最後一個數字後無空白；若 N 個被選擇數字都不能整除 S，就輸出-1。

範例一：輸入

```
3 2
1 5
6 4
1 1
```

範例一：正確輸出

```
12
6 1
```

（說明）挑選的數字依序是 5, 6, 1，總和 S=12。而此三數中可整除 S 的是 6 與 1，6 在第二群，1 在第 3 群所以先輸出 6 再輸出 1。注意，1 雖然也出現在第一群，但她不是第一群中挑出的數字，所以順序是先 6 後 1。

範例二：輸入

```
4 3
6 3 2
2 7 9
4 7 1
9 5 3
```

範例二：正確輸出

```
31
-1
```

（說明）挑選的數字依序是 6,9,7,9，總和 S=31。而此四數中沒有可整除 S 的，所以第二行輸出-1。

解題思路

- 讀取資料
  - N M：群 個
  - 讀取N次，切割為M筆資料，將每一群反向排序後，加入二維集合 Nums
- 處理資料
  - 取得每一群中的最大值：取索引0，加入LNums集合
  - 最大值(LNums集合)加總和為S
  - 最大值(LNums集合)中可將S整除的數字加入RNums 集合
- 輸出
  - S
  - 最大值中可將S整除的數字(RNums 集合)，無資料顯示-1

解題思路

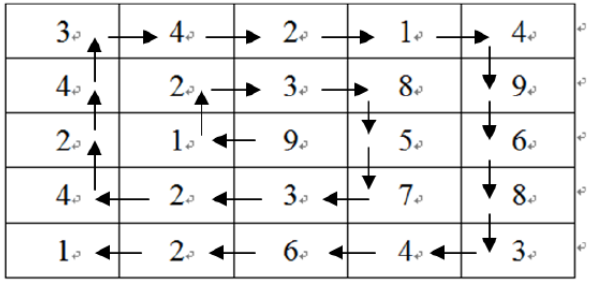
- 讀取資料
  - N M：群 個
  - 讀取N次，切割為M筆資料，將每一群反向排序後，加入二維集合 Nums
- 處理資料
  - 取得每一群中的最大值：取索引0，加入LNums集合
  - 最大值(LNums集合)加總和為S
  - 最大值(LNums集合)中可將S整除的數字加入RNums 集合
- 輸出
  - S
  - 最大值中可將S整除的數字(RNums 集合)，無資料顯示-1

實作題

第 3 題　數字龍捲風

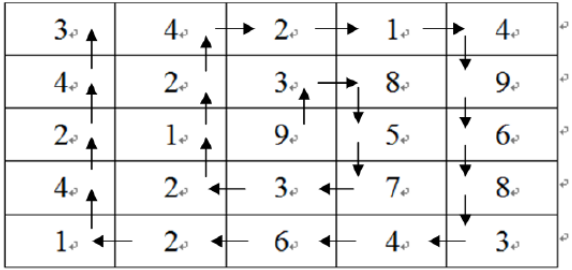
問題描述

給定一個 N*N 的二維陣列，其中 N 是奇數，我們可以從正中間的位置開始，以順時針旋轉的方式走訪每個陣列元素恰好一次。對於給定的陣列內容與起始方向，請輸出走訪順序之內容。下面的例子顯示了 N=5 且第一步往左的走訪順序：



依此順序輸出陣列內容則可以得到「9123857324243421496834621」。

類似地，如果是第一步向上，則走訪順序如下：



依此順序輸出陣列內容則可以得到「9385732124214968346214243」。

輸入格式

輸入第一行是整數 N，N 為奇數且不小於 3。第二行是一個 0~3 的整數代表起始方向，其中 0 代表左、1 代表上、2 代表右、3 代表下。第三行開始 N 行是陣列內容，順序是由上而下，由左至右，陣列的內容為 0~9 的整數，同一行數字中間以一個空白間隔。

輸出格式

請輸出走訪順序的陣列內容，該答案會是一連串的數字，數字之間**不要輸出空白**，結尾有換行符號。

範例一：輸入

5
0
3 4 2 1 4
4 2 3 8 9
2 1 9 5 6
4 2 3 7 8
1 2 6 4 3

範例一：正確輸出

912385732424342149683462 1

範例二：輸入

3
1
4 1 2
3 0 5
6 7 8

範例二：正確輸出

012587634

評分說明

輸入包含若干筆測試資料，每一筆測試資料的執行時間限制(time limit)均為 1 秒，依正確通過測資筆數給分。其中：

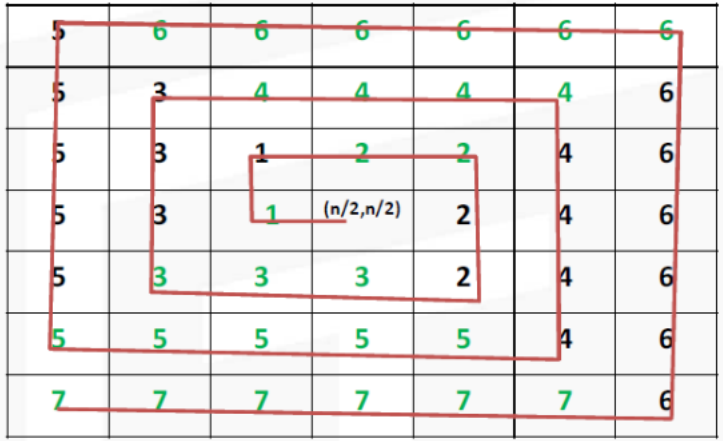
第 1 子題組 20 分，$3 \le N \le 5$，且起始方向均為向左。

第 2 子題組 80 分，$3 \le N \le 49$，起始方向無限定。

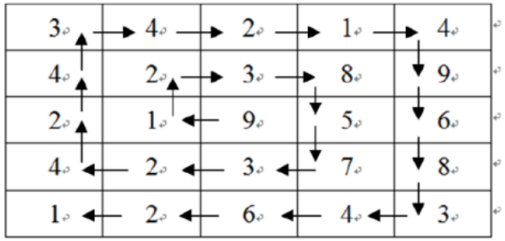
提示：本題有多種處理方式，其中之一是觀察每次轉向與走的步數。例如，起始方向是向左時，前幾步的走法是：左 1、上 1、右 2、下 2、左 3、上 3、……一直到出界為止。

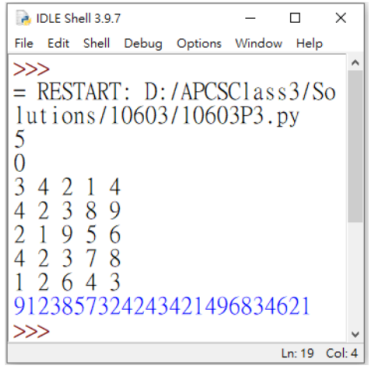
解題思路

- 建立N*N的二維陣列
  - 初始行進方向：0>左、1>上、2>右、3>下
- 觀察：走的方向及走的步數
  - 由[N//2, N//2]位置開始
  - 左1>上1>右2>下2>左3>上3>右4>下4
  - 步數為由1步開始，每2次改變後遞增
  - 方向順序為[左,上,右,下]
    - [[0,-1], [-1,0], [0,1], [1,0]]



```python
direction=[[0,-1],[-1,0],[0,1],[1,0]]          #方向順序[左、上、右、下]
n = int(input())
d = int(input())                               #初始方向: 0代表左、1代表上、2代表右、3代表下

data=[]
for i in range(n):
    temp=[]
    temp=input().split()
    data.append(temp)

step = 1
stepcounter = 0
counter = 1
row = n // 2
col = n // 2
print("%d" %int(data[row][col]),end='')
while counter < n * n:
    for i in range(step):
        row += direction[d][0]
        col += direction[d][1]
        print("%d" %int(data[row][col]),end='')
        counter=counter+1
        if counter==n*n: # 超過最大數量跳出迴圈
            break
    if counter==n*n:# 超過最大數量跳出迴圈
        break
    stepcounter=stepcounter+1#步驟數量+1
    if stepcounter % 2 == 0: #步驟數量 如是偶數就改變動作次數
        step=step+1
    d += 1 #切換到下一個動作
    d %= 4 #每四筆換到0
```





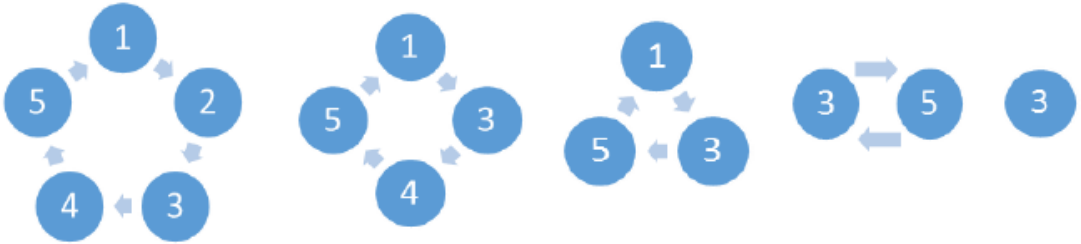
實作題

第 3 題　定時 K 彈

問題描述

「定時 K 彈」是一個團康遊戲，N 個人圍成一個圈，由 1 號依序到 N 號，從 1 號開始依序傳遞一枚玩具炸彈，炸彈每次到第 M 個人就會爆炸，此人即淘汰，被淘汰的人要離開圓圈，然後炸彈再從該淘汰者的下一個開始傳遞。遊戲之所以稱 K 彈是因為這枚炸彈只會爆炸 K 次，在第 K 次爆炸後，遊戲即停止，而此時在第 K 個淘汰者的下一位遊戲者被稱為幸運者，通常就會被要求表演節目。例如 N=5，M=2，如果 K=2，炸彈會爆炸兩次，被爆炸淘汰的順序依序是 2 與 4（參見下圖），這時 5 號就是幸運者。如果 K=3，剛才的遊戲會繼續，第三個淘汰的是 1 號，所以幸運者是 3 號。如果 K=4，下一輪淘汰 5 號，所以 3 號是幸運者。

給定 N、M 與 K，請寫程式計算出誰是幸運者。



輸入格式

輸入只有一行包含三個正整數，依序為 N、M 與 K，兩數中間有一個空格分開。其中 $1 \le K < N$。

輸出格式

請輸出幸運者的號碼，結尾有換行符號。

範例一：輸入

5 2 4

範例一：正確輸出

3

（說明）

被淘汰的順序是 2、4、1、5，此時 5 的下一位是 3，也是最後剩下的，所以幸運者是 3。

範例二：輸入

8 3 6

範例二：正確輸出

4

（說明）

被淘汰的順序是 3、6、1、5、2、8，此時 8 的下一位是 4，所以幸運者是 4。

評分說明

輸入包含若干筆測試資料，每一筆測試資料的執行時間限制(time limit)均為 1 秒，依正確通過測資筆數給分。其中：

第 1 子題組 20 分，$1 \le N \le 100$，且 $1 \le M \le 10$，$K = N-1$。

第 2 子題組 30 分，$1 \le N \le 10{,}000$，且 $1 \le M \le 1{,}000{,}000$，$K = N-1$。

第 3 子題組 20 分，$1 \le N \le 200{,}000$，且 $1 \le M \le 1{,}000{,}000$，$K = N-1$。

第 4 子題組 30 分，$1 \le N \le 200{,}000$，且 $1 \le M \le 1{,}000{,}000$，$1 \le K < N$。

- 讀取輸入 N M K
  - 切割、轉型
  - 建立pList，索引值為1-N
  - 索引idx由0開始
  - 執行遊戲K次
    - 每次遊戲傳遞炸彈M次爆炸，索引為(idx+M-1)%len(pList)
    - 彈出爆炸的玩家， pList.pop(idx)
  - 取出目前索引指向的物件
    - pList[idx%len(pList)]

10510P3-2.py - D:\APCSClass3\Solutions\10510\10510P3-2.py (3.9.7)

File Edit Format Run Options Window Help

```python
def josephusLuckyPerson(ls, m, k):
    idx = 0
    for i in range(k):
        idx = (idx+m-1)%len(ls)
        ls.pop(idx)
    return ls[idx%len(ls)]


#讀取輸入 N M K, 切割、轉型
inStrs = input().split(' ')
N = int(inStrs[0])
M = int(inStrs[1])
K = int(inStrs[2])

#建立pList，索引值為1-N
pList=[i for i in range(1, N+1)]
#print(pList)

luckyPerson = josephusLuckyPerson(pList, M, K)
print(luckyPerson)
```

Ln: 21 Col: 0

N=8，M=3,K=6

輪次	原圈	rotate(-2)	淘汰後
1	1,2,3,4,5,6,7,8	3,4,5,6,7,8,1,2	4,5,6,7,8,1,2
2	4,5,6,7,8,1,2	6,7,8,1,2,4,5	7,8,1,2,4,5
3	7,8,1,2,4,5	1,2,4,5,7,8	2,4,5,7,8
4	2,4,5,7,8	5,7,8,2,4	7,8,2,4
5	7,8,2,4	2,4,7,8	4,7,8
6	4,7,8	8,4,7	4,7

最後剩下「4, 7」，隊首4是幸運者！

```python
from collections import deque

N, M, K = map(int, input().split())
people = deque(range(1, N + 1))        # 建立1~N號的環形隊列

for _ in range(K):
    people.rotate(-(M - 1))            # 傳遞炸彈M-1次，讓第M個人到最前面
    people.popleft()                   # 淘汰最前面的人

print(people[0])                       # 淘汰K次後，隊首即為幸運者
```